A Trimester – 1 Class Project Report on

CareQueue: Hospital Emergency Management System

Submitted in the partial fulfillment of the requirements for

26CS5101E – Computational Thinking and Programming

Master of Technology In

Artificial Intelligence & Data Science

By

Student ID:2601150056
Student Name: M Geetha Krishna Venkatesh

under the supervision of

Dr. G. Syam Prasad

Professor



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd., u/s 3 of UGC Act 1956)
Greenfields, Vaddeswaram, Guntur (Dist.), Andhra Pradesh - 522502

2026 – 27

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled "CareQueue: Hospital Emergency Management System" is being submitted by Geetha Krishna Venkatesh Maroju with Registered ID:2601150056 submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University is a record of Bonafide work carried out under our guidance and supervision during the academic year 2026 – 2027.

Dr. G. Syam Prasad

Project Supervisor

Signature

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DECLARATION

The Trimester -1 Class Project Report entitled "CareQueue: Hospital Emergency Management System" is a record of Bonafide work of Geetha Krishna Venkatesh Maroju with Registered ID: 2601150056 submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University.

Geetha Krishna Venkatesh Maroju

ID: 2601150056

ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the trimester - 1 class project report.

I express sincere gratitude to HOD of AI&DS Dept. **Dr. M. Venkata Naresh** for his leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. G. Syam Prasad** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

ABSTRACT

CareQueue is a web-based Hospital Emergency Management System developed as a class project for Computational Thinking and Programming. It models a simplified emergency-department workflow by combining patient registration, a severity based waiting queue, doctor and bed availability tracking, treatment status updates, and treatment history in one interface. The application uses Python with Flask for request handling and SQLite for local persistence.

The project demonstrates problem decomposition, abstraction, algorithm and data-structure selection, database operations, input validation, modular design, and test planning. The queue ordering rule places Critical cases before Urgent and Normal cases, then uses registration time to break ties. The report also evaluates Python programming techniques, object-oriented and functional design, concurrency choices, software testing, DevOps practices, and responsible AI-assisted development in relation to the project.

CareQueue is an educational prototype. Its severity labels are demonstration categories and are not a clinical triage protocol. The application must not be used to diagnose patients or make real medical decisions.

Keywords: Computational Thinking, Python, Flask, SQLite, Priority Scheduling, Software Engineering, Testing.

TABLE OF CONTENTS

Section	Title
1	Introduction
2	Problem Statement, Objectives and Scope
3	Requirements and Technology Stack
4	System Architecture and Data Design
5	Computational Thinking and Algorithm Design
6	Python Programming Concepts and Data Structures
7	Software Design and Engineering
8	Functional Logic, Persistence and Concurrency Considerations
9	Implementation and Code Snippets
10	User Interface and Demonstration Screenshots
11	Testing and Quality Engineering
12	Results and Discussion
13	Limitations and Future Scope
14	Conclusion
References	References
Appendix A	Representative Test Inventory

1. INTRODUCTION

Emergency departments must coordinate incoming patients, urgency categories, staff availability, beds, and treatment records. When information is scattered across separate records or managed manually, staff may find it harder to see the current queue and resources in one place.

CareQueue provides a small web-based demonstration of this workflow. The dashboard summarizes active cases and resources; a registration form captures patient details; the queue displays waiting cases by severity and arrival order; and the treatment/history page records workflow progress. The project focuses on programming and software engineering rather than clinical decision-making.

1.1Project Overview:

Item	Description
Project name	CareQueue: Hospital Emergency Management System
Course	26CS5101E – Computational Thinking and Programming
Application type	Local web application / educational prototype
Backend	Python and Flask
Persistence	SQLite relational database
Interface	HTML, CSS and JavaScript templates/assets
Core workflow	Patient intake, priority queue, resource allocation, treatment completion and history

1.2 Scope and Boundary:

The scope includes patient intake, queue ordering, search/filter controls, doctor and bed status tracking, treatment start/completion, history viewing, dashboard indicators, and CSV export as represented in the project interface. Authentication, multi-hospital deployment, production security controls, integration with hospital systems, and clinical triage logic are not assumed to be implemented unless confirmed in the final source.

1.3 Intended Use:

The system is intended for academic demonstration using synthetic or authorized sample data. It is not a medical device, does not diagnose patients, and does not replace professional clinical judgment or approved hospital protocols.

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

2.1 Problem Statement

A simplified emergency workflow needs to track several waiting patients with different urgency labels while showing whether doctors and beds are available. The project problem is to build a centralized prototype that stores patient records, presents a predictable waiting order, supports treatment status changes, and maintains a history of completed cases.

2.2 Objectives

- Register patients with basic details, symptoms, and a severity category.
- Order waiting records by severity and arrival time.
- Present dashboard indicators for the queue and available resources.
- Track doctor and bed statuses for the demonstration workflow.
- Record treatment start, completion, outcome, and notes.
- Search and review treatment history and export records where supported by the application.
- Demonstrate computational thinking, Python programming, persistence, and software quality practices.

2.3 Problem Decomposition

Subproblem	Responsibility	Expected output
Patient intake	Validate and store form details	Patient record
Queue management	Order active waiting cases	Priority-aware queue
Resource tracking	Represent doctor and bed statuses	Availability summary
Treatment lifecycle	Update treatment state and timeline	Current status and history
Reporting	Present history and export records	Reviewable records/CSV

3. REQUIREMENTS AND TECHNOLOGY STACK

3.1 Functional Requirements

ID	Requirement
FR-01	Accept patient registration and validate required values.
FR-02	Persist patient details and workflow status in SQLite.
FR-03	Display waiting patients in deterministic severity and arrival order.
FR-04	Support search and severity filtering in queue management.
FR-05	Show and update doctor and bed availability/status.
FR-06	Maintain inventory quantities/status for five equipment classes.
FR-07	Start treatment by selecting a waiting patient and available resources.
FR-08	Complete treatment, store outcome/notes/timestamps, and release assigned resources.
FR-09	Show dashboard indicators, active alerts and visual summaries.
FR-10	Display a separate live queue board.
FR-11	Show treatment history and export records as CSV.

3.2 Non-Functional Requirements

- Usability: key queue and resource information should be visible without navigating through many screens.
- Maintainability: separate request handling, data access, and presentation responsibilities where practical.
- Reliability: validate user inputs and handle invalid or missing records.
- Persistence: store data beyond a single request using SQLite.
- Privacy: use synthetic or authorized data and avoid exposing unnecessary personal information.
- Performance: keep queue sorting and page rendering reasonable for a small academic dataset.

3.3 Technology Stack

Technology	Role in CareQueue	Reason for selection
Python	Application and workflow logic	Readable syntax and standard data structures.
Flask	HTTP routes and rendering	Lightweight framework suitable for a local web prototype.
SQLite	Relational persistence	Local file-based relational database.
HTML/CSS/JavaScript	Interface and interactions	Forms, page layout, responsive visual presentation and browser interactions.
pytest	Automated testing	Repeatable route and helper tests.
Hypothesis plugin	Available in the test environment	Property-based testing support is installed; only claim property-based test execution if tests explicitly use it.

Environment reported by the supplied local test output: macOS, Python 3.9.21, pytest 8.4.2, and Hypothesis 6.141.1.

4. SYSTEM ARCHITECTURE AND DATA DESIGN

4.1 Architecture

The browser displays the dashboard and forms. A Flask route receives each request, validates input, invokes workflow logic, and reads or writes data in SQLite. Results are passed to the relevant template, which renders the updated state.

```
Browser / HTML forms
        |
        | HTTP request
        v
Flask routes and validation
        |
        +---- Queue / treatment workflow logic
        |
        v
SQLite database
        |
        v
Query results -> templates -> browser
```

4.2 Main Modules

Module	Purpose
Dashboard	Summary indicators, alerts, queue and visual summaries.
Patient intake	Collects patient fields and validates submitted values.
Queue	Retrieves active waiting cases and orders them.
Resource status	Displays and updates doctor/bed states.
Treatment	Tracks treatment start, completion, outcome and notes.
History/export	Shows past cases and supports CSV export where available.

4.3 Logical Data Model

The following is a logical model for documenting the workflow. The final report should use the exact table names and columns found in the actual SQLite database.

Entity	Key information represented	Purpose
Patient	ID, name, phone, age, severity, symptoms, status and timestamps	Registration, queue ordering and treatment lifecycle.
Doctor	ID, name, specialty and status	Availability and assignment tracking.
Bed	ID, bed name and status	Availability and allocation tracking.
General resource	Resource name, description, quantity and status	Equipment inventory management.
Patient-resource assignment	Patient, resource, quantity, assigned/released timestamps	Tracks equipment assignment and release.

5. COMPUTATIONAL THINKING AND ALGORITHM DESIGN

5.1 Decomposition

The problem is separated into patient registration, queue management, resource tracking, treatment management, history reporting and dashboard visualization. This reduces the complexity of the overall application because each part has a clear responsibility and can be tested through specific scenarios.

5.2 Abstraction and Pattern Recognition

Patient records retain the fields needed for the demonstration workflow. Severity is represented by numeric codes mapped to Critical, Urgent and Normal labels. Doctors, beds and equipment are represented through inventory records and status values rather than detailed real-world clinical models. Keyword rules identify illustrative specialty and equipment suggestions from entered symptoms.

5.3 Algorithm Selection: Priority Scheduling

A priority-aware ordering rule fits the demonstration because not all waiting records have the same assigned severity category. Critical cases are ranked before Urgent cases, which are ranked before Normal cases. Registration time is used as a tie-breaker within the same category. These labels are not clinical recommendations and must not be interpreted as a validated triage protocol.

```
SEVERITY_RANK = {
    "Critical": 0,
    "Urgent": 1,
    "Normal": 2,
}
def queue_key(patient):
    return (
        SEVERITY_RANK.get(patient.get("severity"), 99),
        patient.get("created_at", "")
    )
```

5.4 Divide and Conquer: Merge Sort

The application includes a custom generic Merge Sort helper. It divides the input list into two halves, recursively sorts each half, and merges the sorted halves. When two keys are equal, the implementation selects the left-side element first, preserving stable ordering. The helper returns a new list and does not mutate the original input list.

```
def merge_sort(items: list[T], key: Callable[[T], Any]) -> list[T]:
    if len(items) <= 1:
        return items.copy()
    middle = len(items) // 2
    left = merge_sort(items[:middle], key)
```

```
right = merge_sort(items[middle:], key)
merged: list[T] = []
i = j = 0
while i < len(left) and j < len(right):
    if key(left[i]) <= key(right[j]):
        merged.append(left[i])
        i += 1
    else:
        merged.append(right[j])
        j += 1
merged.extend(left[i:])
merged.extend(right[j:])
return merged
```

For n records, Merge Sort has O(n log n) time complexity in the usual analysis and O(n) auxiliary storage for the merged output and recursive processing. The project includes an automated test that verifies priority ordering and stable ties.

5.5 Greedy-Style Resource Allocation

When the operator starts the next treatment, the route chooses the highest-priority waiting patient, then selects a suitable available doctor and the first available bed. It checks the configured demonstration equipment rules and reserves equipment only when the inventory is available and has an unallocated unit. This is a local greedy-style decision process; it is not a proof of globally optimal resource allocation.

5.6 Rule-Based Suggestions

Doctor specialty and equipment suggestions are produced by transparent keyword rules. These are ordinary programmed rules, not a trained artificial-intelligence model and not clinical advice. The user interface explicitly states that staff must confirm equipment needs.

5.7 Complexity Summary

Operation	Expected/typical complexity	Notes
Custom Merge Sort	O(n log n) time; O(n) auxiliary space	Used for deterministic ordering and tested for stable ties.
Render a list of n records	O(n) rendering work, excluding database/template overhead	Every displayed record must be processed.
Severity label lookup	O(1) average for dictionary lookup	Small fixed mapping.
Database query/update	Depends on query plan, indexes and row count	No measured database benchmark is claimed.
CSV row generation	O(1) extra row-generation state per yielded row, excluding source rows and final response buffer	Generator avoids a second full list of formatted rows; response is buffered.

6. PYTHON PROGRAMMING CONCEPTS AND DATA STRUCTURES

6.1 Data Structures Used

- Dictionary: maps severity labels to priority ranks.
- List: holds retrieved records for display and sorting.
- Tuple: combines severity rank and arrival time into a deterministic sort key.
- Relational tables: persist patient and treatment information.

6.2 Type Hints and Generic Programming

The Merge Sort helper uses a TypeVar and type annotations to accept lists of different record types, with a caller-supplied key function.

```
T = TypeVar("T")

def merge_sort(
    items: list[T],
    key: Callable[[T], Any]
) -> list[T]:
    ...
```

Type hints communicate expected values to readers and static analysis tools. They do not replace validation of untrusted form inputs.

6.3 Generator and Iteration

The CSV export helper is implemented as a generator. It yields the header row first and then yields each patient's export values. The CSV writer consumes the yielded rows sequentially. This avoids creating an additional in-memory list containing every formatted export row, although the complete HTTP response is still buffered.

```
def iter_csv_rows(rows):
    yield ["Patient ID","Name","Phone","Age","Severity","Symptoms","Status","Registered at",
           "Treatment started","Treatment completed","Doctor","Bed","Outcome","Notes"]
    for r in rows:
        yield
[r["id"],r["name"],r["phone"],r["age"],LABELS.get(r["severity"],"Unknown"),r["symptoms"],

r["status"],r["created_at"],r["started_at"],r["completed_at"],r["doctor"],r["bed_name"],r["outcome"],r
["notes"]]
```

6.4 Database Context and Resource Cleanup

Database connections are closed in cleanup paths. Treatment start and completion use transaction control to keep related changes consistent. This is important because a patient status, doctor status, bed status and equipment assignment may all change as part of one workflow operation.

6.5 Validation

The application checks required patient details, age bounds, severity codes, phone-number format and duplicate active patients. Validation is performed in the server-side workflow and automated tests verify representative invalid cases. Browser-side required fields improve usability but are not treated as a replacement for server-side checks.

7. SOFTWARE DESIGN AND ENGINEERING

7.1 Modular Design

The application is easier to maintain when HTTP routing, database access, queue ordering, and HTML presentation have clear responsibilities. Route functions should validate requests and coordinate operations, while reusable helper functions should contain queue and data-processing rules.

7.2 Object-Oriented Design Concepts

The current project is described as a Flask application; this report does not assume that it uses a class hierarchy. The following small class is an illustrative model for explaining encapsulation and type hints, and should only be presented as implemented if the source contains a matching design.

```
from dataclasses import dataclass
from datetime import datetime

@dataclass
class Patient:
    patient_id: int
    name: str
    severity: str
    created_at: datetime
    status: str = "Waiting"
```

Encapsulation groups related data and behavior. Composition can represent a treatment record referring to a patient. Inheritance is useful when types share a genuine common contract, but should not be added simply to satisfy a checklist. Abstraction hides implementation details behind a small interface.

7.3 Type Hints and Generic Programming

Type hints document expected values and enable static analysis tools to detect some mistakes before runtime. Generic types such as list[Patient] or dict[str, int] make data structures more explicit. Type hints do not replace runtime validation of untrusted form data.

7.4 Input Validation and Error Handling

Validation rejects invalid ages, invalid severity values, malformed phone numbers, and duplicate active patient entries. The workflow also handles the absence of waiting patients and unavailable doctors or beds by rolling back the allocation attempt and returning an explanatory message. Editing and deletion are limited to waiting patients, preventing certain changes while a patient is already in treatment.

7.5 Privacy and Configuration

Demonstrations should use synthetic or authorized records. The source contains a development fallback secret key; a deployed version should supply a strong secret through environment configuration and add authentication, access control, deployment hardening and privacy protections before handling real patient data.

8. FUNCTIONAL LOGIC, PERSISTENCE AND CONCURRENCY CONSIDERATIONS

8.1 Functional Logic

The queue helper takes input records and a key function and returns an ordered list, separating the sorting operation from the data source. This resembles a functional style because the helper does not mutate the original list. The key function is passed as an argument, which is an example of a higher-order function**.**

8.2 Synchronous Request Model

The application uses a conventional synchronous Flask request model. For a local educational workflow involving form submissions, short database queries and page rendering, this approach is sufficient. The current code does not claim to implement threading, multiprocessing or asyncio.

8.3 Future Concurrency Considerations

If the application is expanded to multiple workers or more concurrent users, resource assignment needs careful transaction handling and tests for race conditions. Authentication, authorization, database locking, deployment configuration and recovery procedures would also require additional design and testing.

Mechanism	Typical use	Status in current project
Threading	I/O-bound concurrent tasks	Not implemented/claimed.
Multiprocessing	Separate processes for CPU-bound work	Not implemented/claimed.
Asyncio	Concurrent I/O with async-compatible libraries	Not implemented/claimed.
SQLite transactions	Consistent related database changes	Used for treatment allocation/completion.

9. IMPLEMENTATION AND CODE SNIPPETS

9.1 Application Factory and Configuration

The application exposes a create_app function and allows configuration overrides, which supports isolated test databases. The tests supply a temporary database path and enable Flask testing mode, preventing tests from depending on the normal local database.

```
def create_app(config: Optional[dict[str, Any]] = None) -> Flask:
    app = Flask(__name__)
    app.config.update(
        SECRET_KEY=os.environ.get("SECRET_KEY", "development-key-change-me"),
        DATABASE=str(Path(__file__).with_name("hospital.db")),
    )
    if config:
        app.config.update(config)
    # Database initialization and routes follow.
    return app
```

Security note: the fallback key is intended only for development. A deployment should provide SECRET_KEY through environment configuration.

9.2 Queue Ordering with Merge Sort

The custom helper recursively divides the list and merges the sorted halves using the provided key. The test suite includes a test named test_merge_sort_orders_priority_and_stable_ties.

```
def merge_sort(items: list[T], key: Callable[[T], Any]) -> list[T]:
    if len(items) <= 1:
        return items.copy()
    middle = len(items) // 2
    left = merge_sort(items[:middle], key)
    right = merge_sort(items[middle:], key)
    merged: list[T] = []
    i = j = 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

9.3 Starting Treatment and Allocating Resources

The route starts a transaction, selects the next waiting patient in severity/time/ID order, checks available doctor and bed records, and then updates the patient and resource statuses. Equipment assignment uses the configured rule list and checks availability before inserting an assignment.

```
@app.post("/treat-next")
def treat_next():
    db = db_connect()
    try:
        db.execute("BEGIN IMMEDIATE")
        patient = db.execute("""
            SELECT id, name, severity, symptoms
            FROM patients
            WHERE status = 'waiting'
            ORDER BY severity ASC, created_at ASC, id ASC
            LIMIT 1
        """).fetchone()
```

```
        if patient is None:
            db.rollback()
            flash("There are no patients waiting.", "error")
            return redirect(url_for("queue_management"))

        # Select an available doctor and bed; reserve eligible equipment.
        # The implementation then updates patient/resource statuses
        # and commits the database transaction.
    finally:
        db.close()
```

9.4 Completing Treatment and Releasing Resources

Completion records the timestamp, outcome and notes, then returns the assigned doctor and bed to available status and marks active equipment assignments as released.

```
db.execute(
    """UPDATE patients
       SET status='treated', completed_at=?, outcome=?, notes=?
       WHERE id=?""",
    (now_iso(), outcome, notes, patient_id),
)
db.execute(
    "UPDATE doctors SET status='available' WHERE id=? AND status='busy'",
    (patient["doctor_id"],),
)
db.execute(
    "UPDATE beds SET status='available' WHERE id=? AND status='occupied'",
    (patient["bed_id"],),
)
db.execute(
    """UPDATE patient_resource_assignments
       SET released_at=?
       WHERE patient_id=? AND released_at IS NULL""",
    (now_iso(), patient_id),
)
db.commit()
```

9.5 Generator-Based CSV Export

The export helper yields a header and one row at a time. The automated test test_csv_export_uses_generator_and_keeps_header checks that export retains its expected header and uses the generator path.

```
def iter_csv_rows(rows):
    yield ["Patient ID","Name","Phone","Age","Severity","Symptoms","Status","Registered at",
           "Treatment started","Treatment completed","Doctor","Bed","Outcome","Notes"]
    for r in rows:
        yield
[r["id"],r["name"],r["phone"],r["age"],LABELS.get(r["severity"],"Unknown"),r["symptoms"],

r["status"],r["created_at"],r["started_at"],r["completed_at"],r["doctor"],r["bed_name"],r["outcome"],r
["notes"]]
```

9.6 Test Example

```
# Representative assertion pattern from the queue tests:
ordered = merge_sort(records, key=queue_key)
assert [key(record) for record in ordered] == sorted(
    key(record) for record in records
)
```

The snippet above illustrates the assertion pattern; the complete automated tests are in tests/test_app.py in the submitted source archive.

10. USER INTERFACE AND DEMONSTRATION SCREENSHOTS

The screenshots below are taken from the supplied latest CareQueue interface. The displayed patient names and values are demonstration records and should not be interpreted as real clinical data.

10.1 Dashboard Overview

The dashboard shows waiting patients, critical cases, available doctors, available and allocated beds, average waiting time and cases treated today. It also displays an active waiting alert and a severity summary.

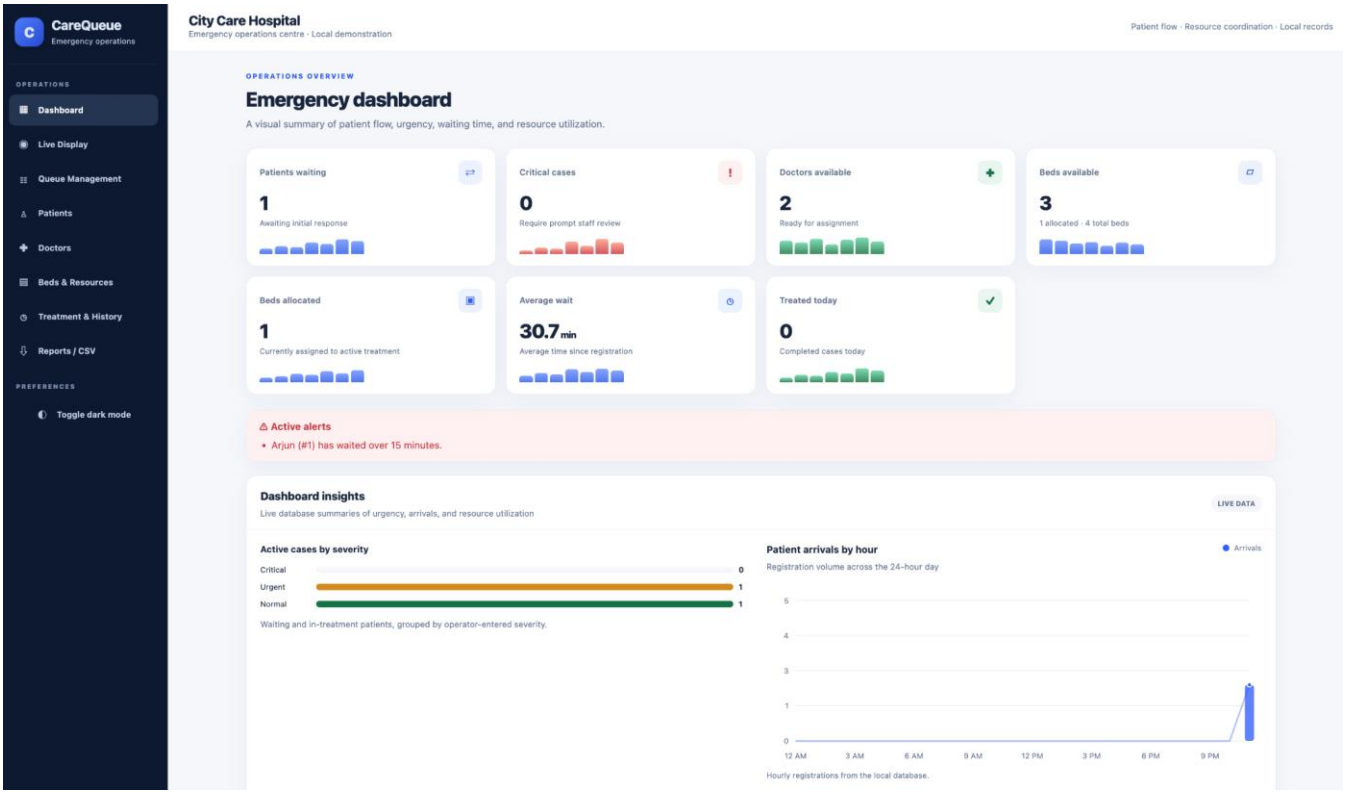


Figure 1. Emergency dashboard with KPI cards, active alert, severity summary and patient arrivals chart.

10.2 Resource Utilization and Inventory Summary

The resource utilization view compares total, allocated and available quantities for beds, wheelchairs, oxygen cylinders, patient monitors, ventilators and IV infusion pumps. Counts are based on active assignments.

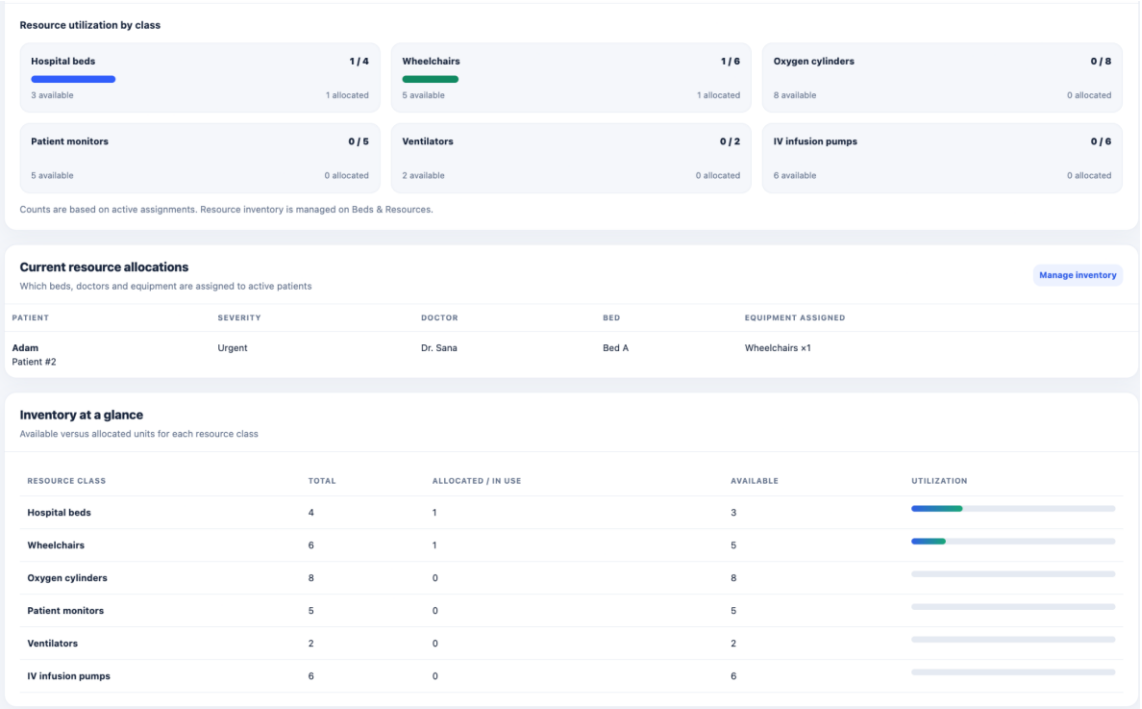


Figure 2. Resource utilization by class and current resource allocation table.

10.3 Waiting-Time Analytics

The waiting-time chart groups recorded registration-to-treatment intervals by registration hour. The current screen includes elapsed time for waiting patients and recorded time to treatment start for patients who have begun treatment. Empty hours are displayed as zero.

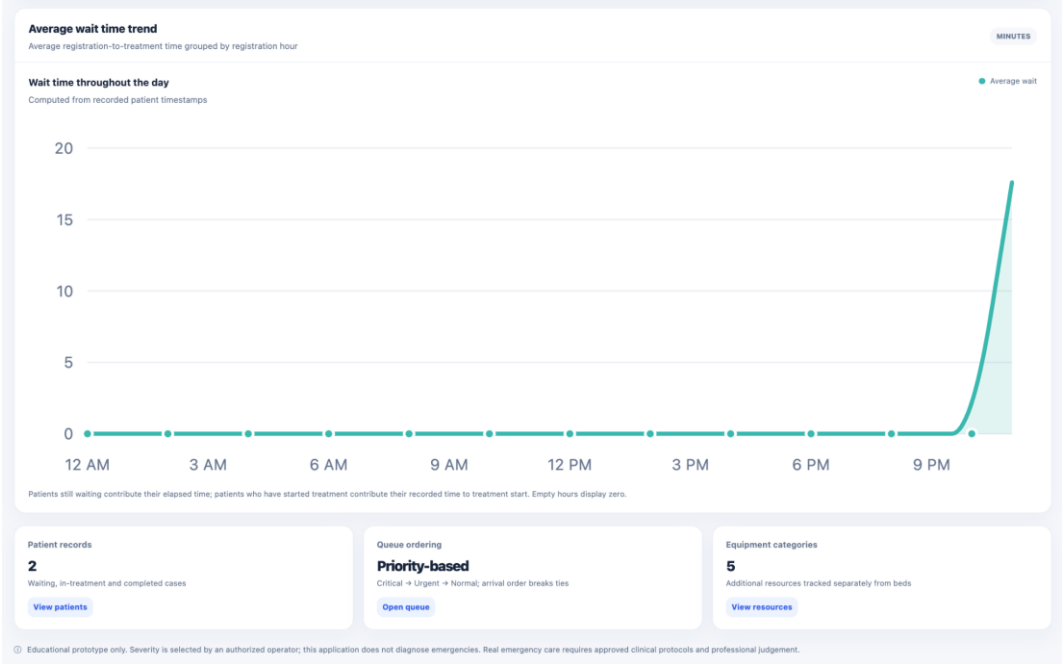


Figure 3. Average waiting-time trend and dashboard insights.

10.4 Priority Queue and Live Display

Queue Management lists waiting patients and provides search and severity filters. The Live Display is a separate page that shows the patient currently being served, the next waiting patient, queue status, and an action to start the next treatment.

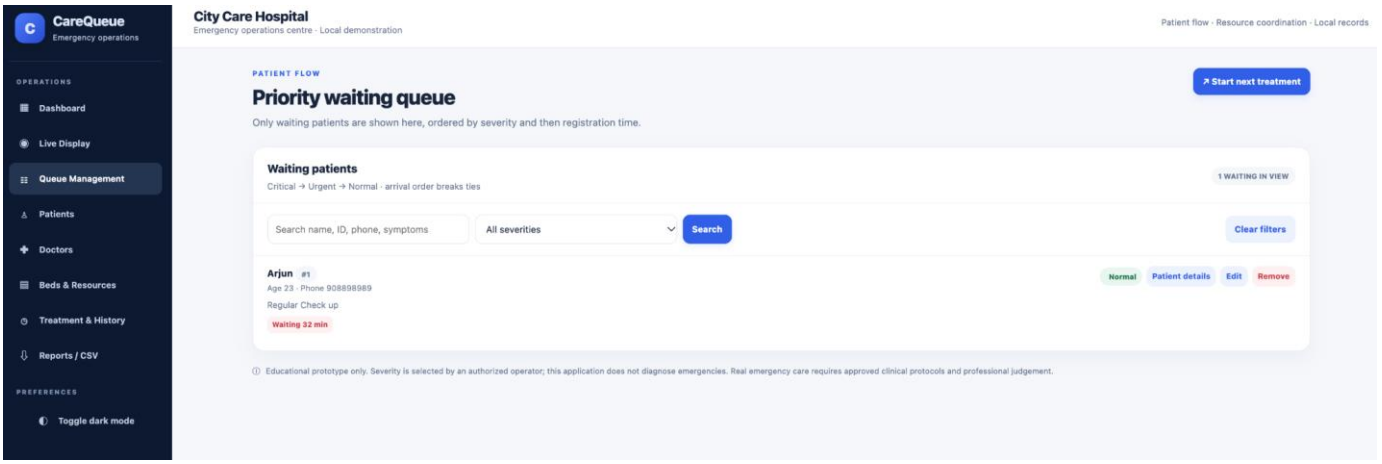


Figure 4. Priority waiting queue with search, severity filtering and patient actions.

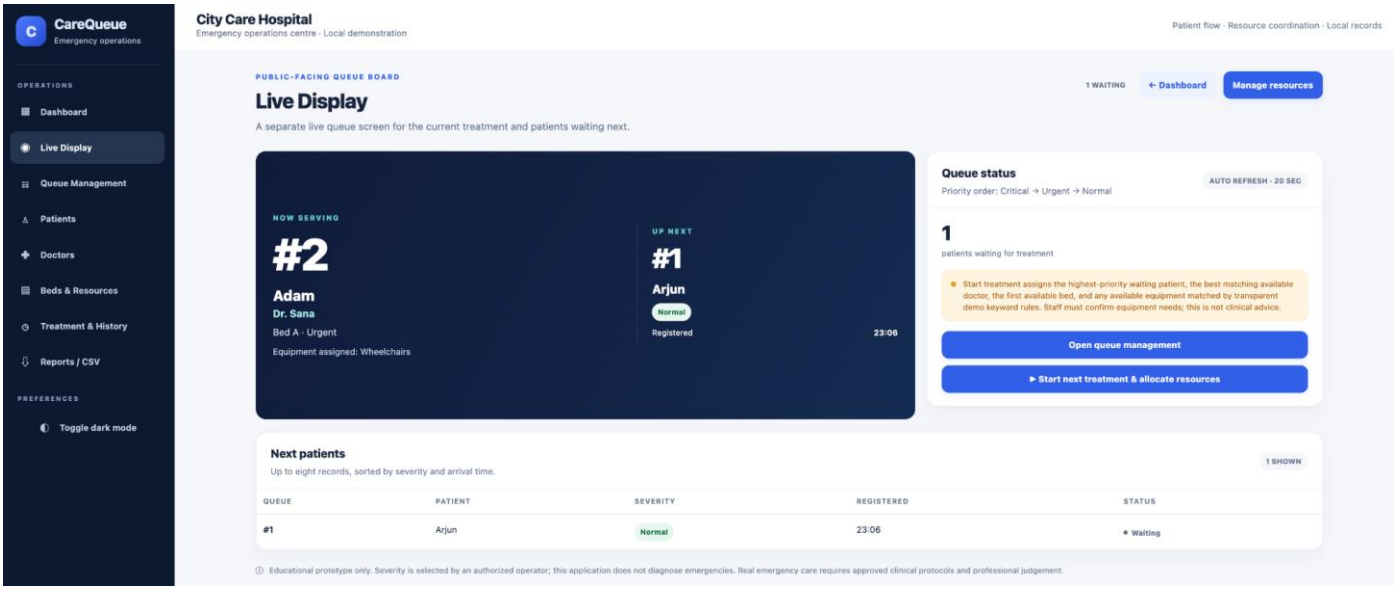


Figure 5. Separate live display showing current treatment, next patient and queue status.

10.5 Patient Registry and Doctor Directory

The patient registry provides registration fields and a table of patient status, severity, waiting time and reason for visit. The doctor directory supports adding doctors with specialties and updating availability.

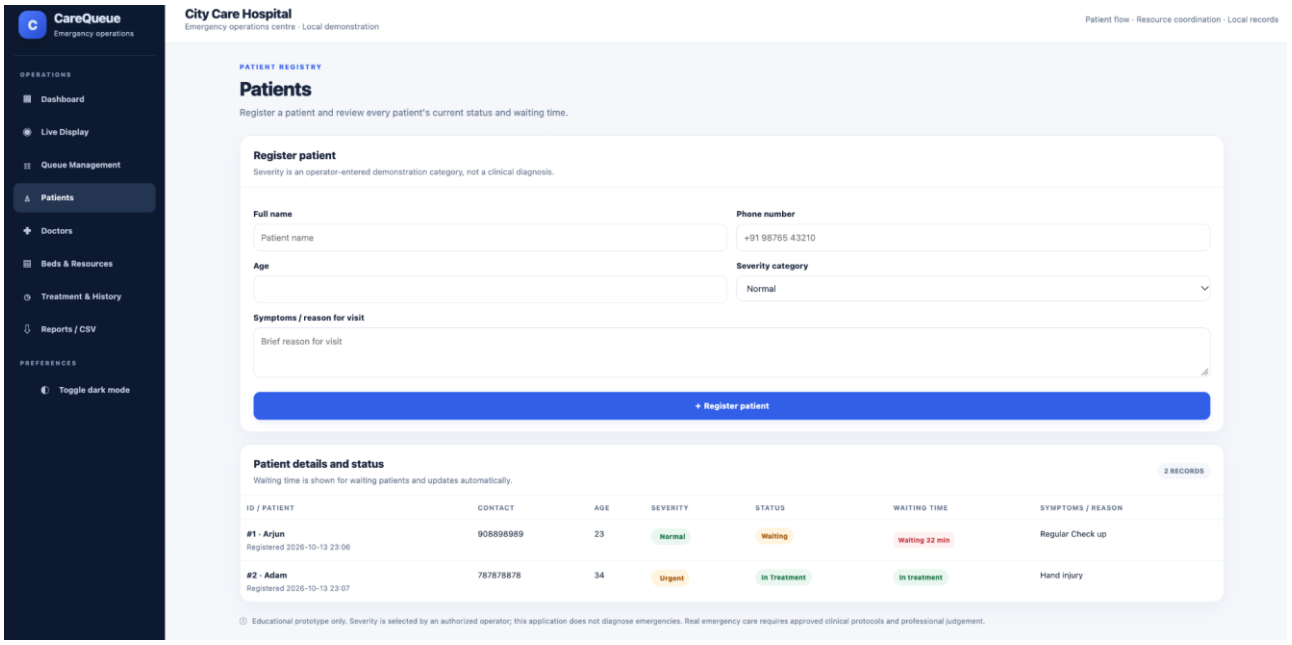


Figure 6. Patient registration form and patient status table.

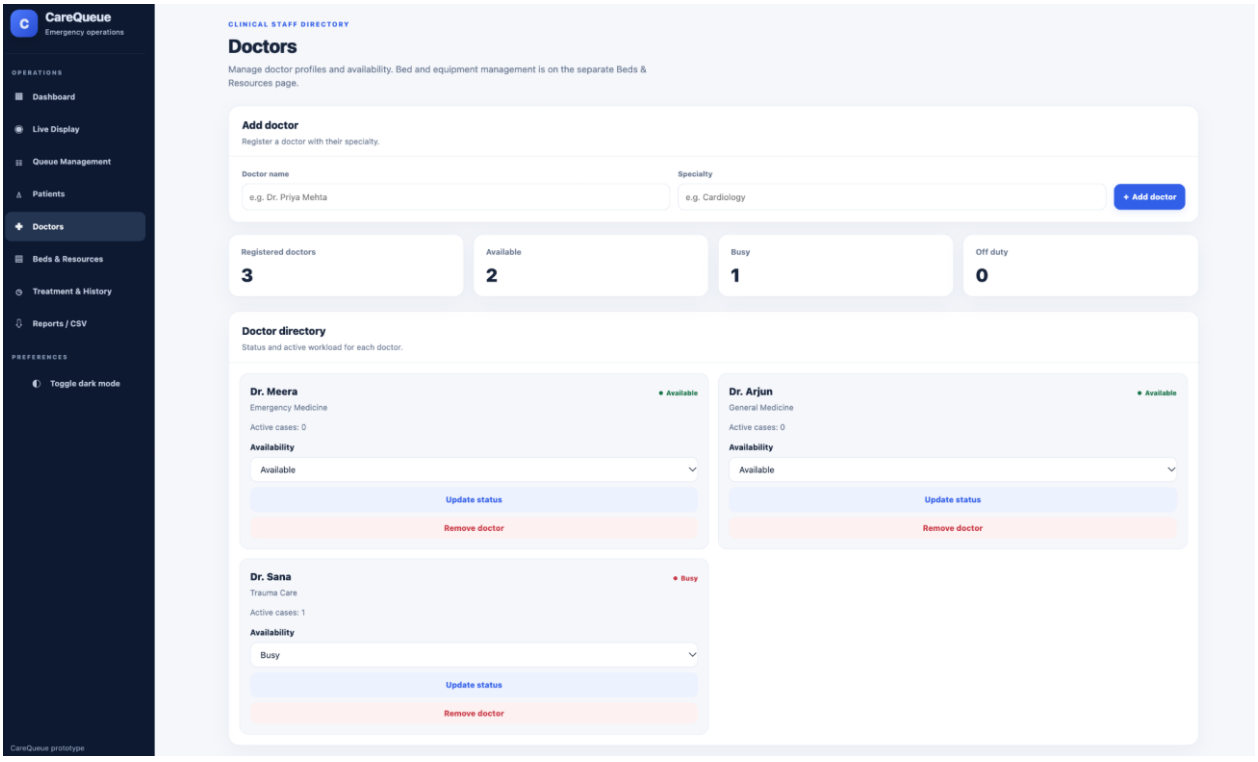


Figure 7. Doctor directory, specialty information and availability controls.

10.6 Beds and Equipment Inventory

Bed Management displays total, available, occupied, reserved and cleaning counts, and provides bed search/filtering, adding, status updates and removal controls. The additional patient-care resource section manages the five equipment categories separately from beds.

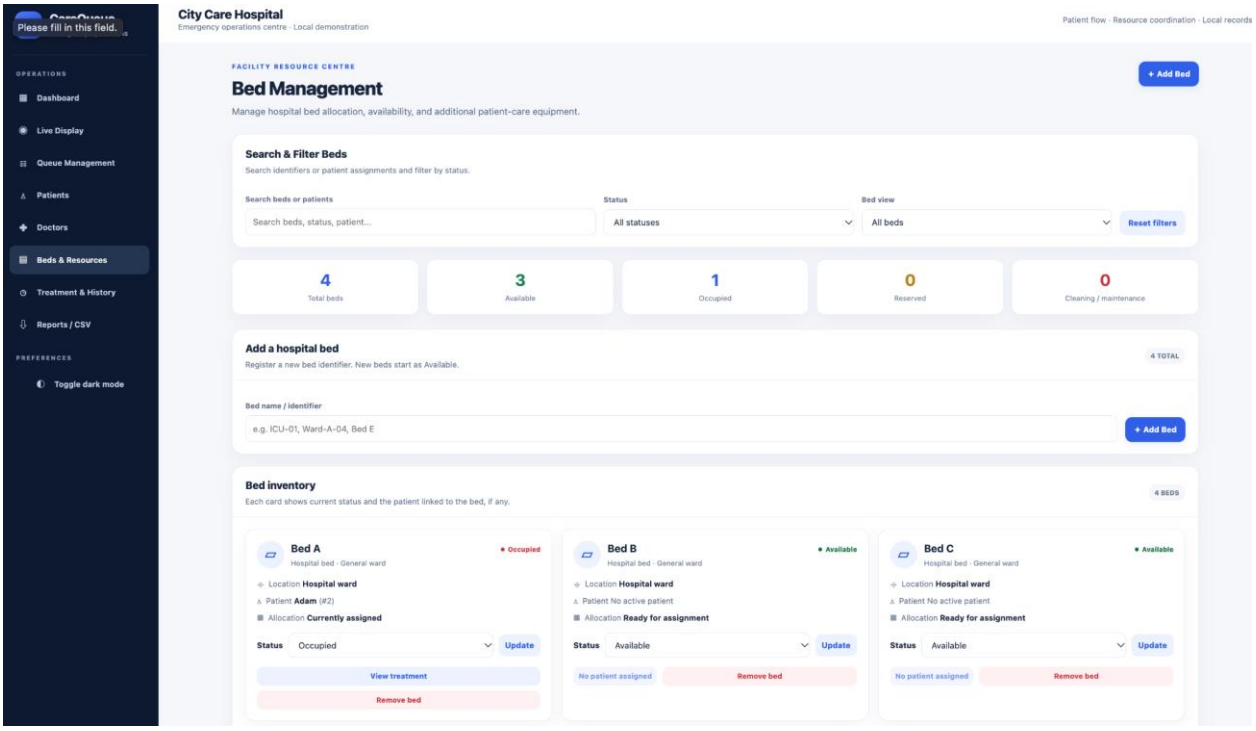


Figure 8. Bed inventory and status management.

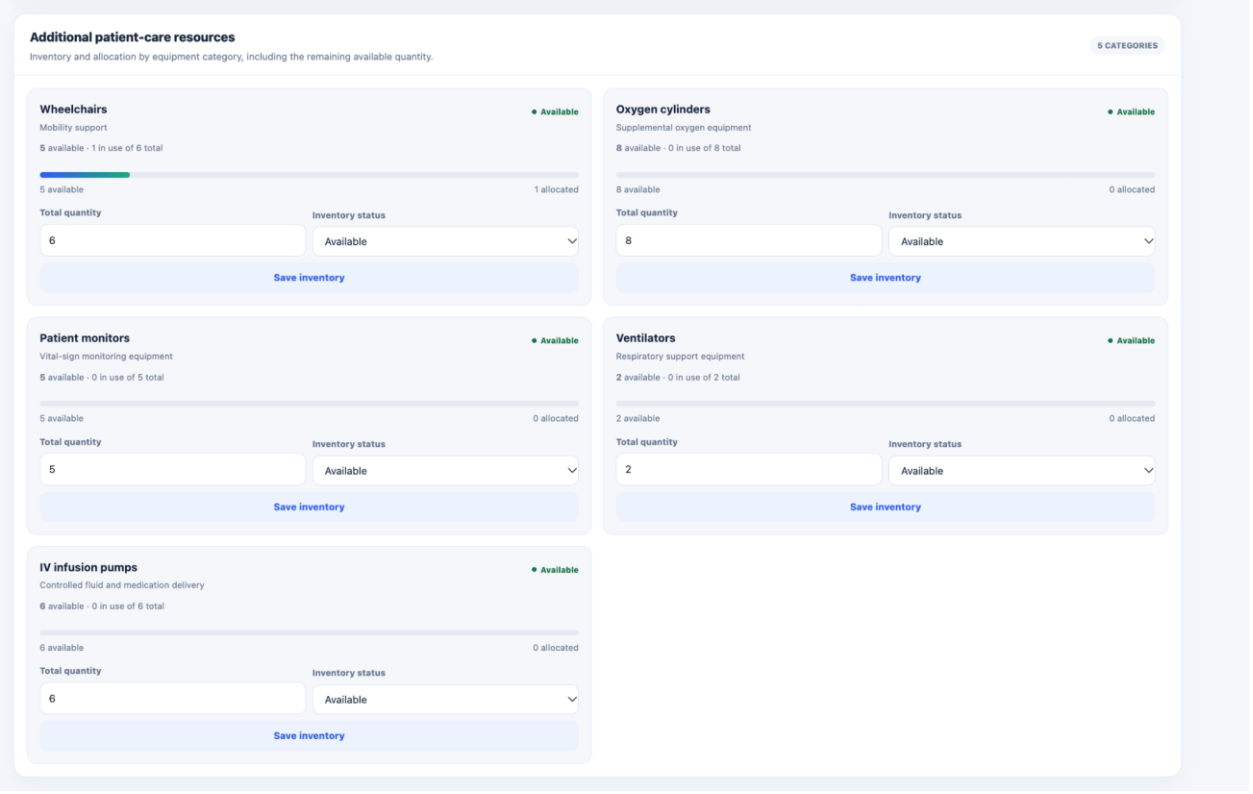


Figure 9. Additional patient-care equipment inventory and quantity/status controls.

10.7 Treatment History and Theme Support

Treatment History displays severity, status and assignment, registration/start/completion timestamps, outcome and notes. The page includes a CSV export action. The interface also supports a dark theme.

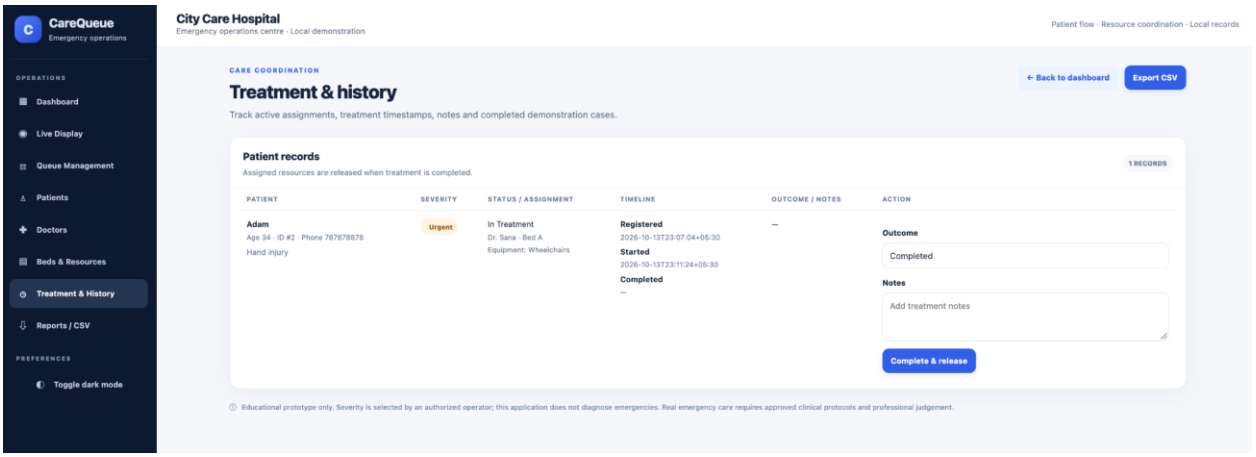


Figure 10. Treatment and history page with timestamps, assignment, outcome and notes.

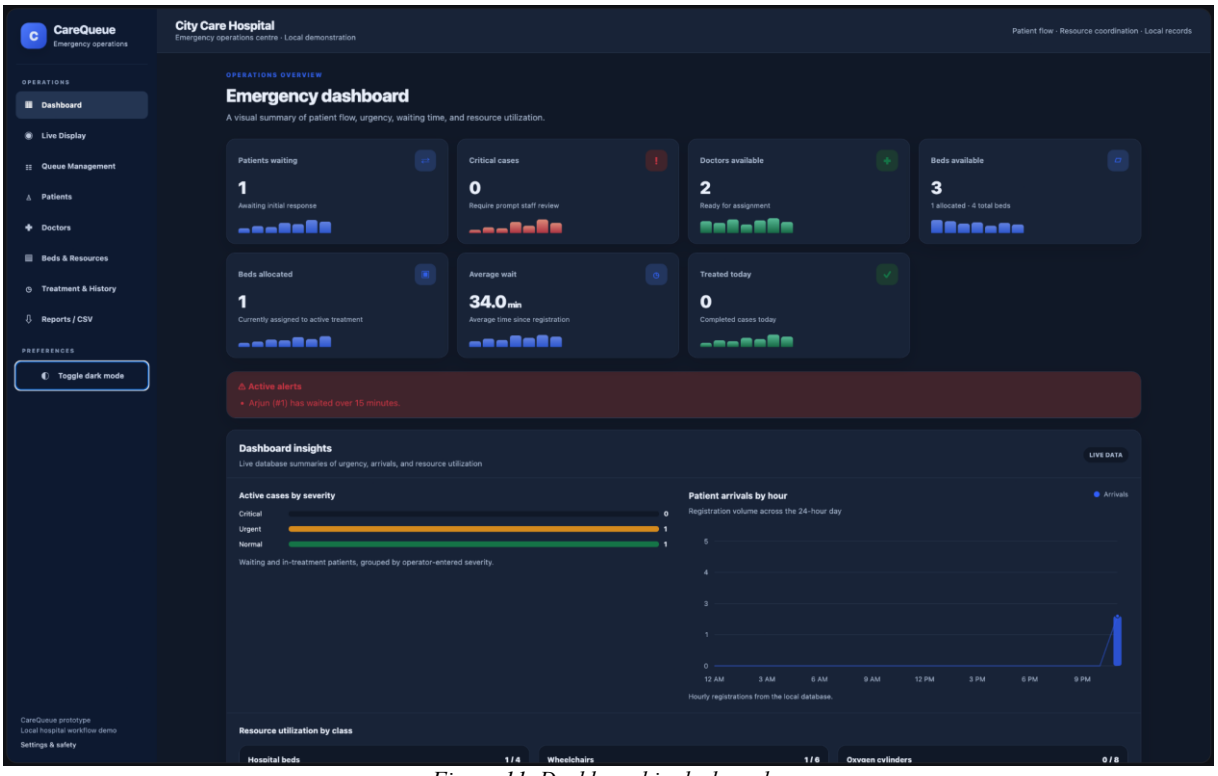


Figure 11. Dashboard in dark mode.

11. TESTING AND QUALITY ENGINEERING

11.1 Test Environment and Summary

The developer ran the automated suite locally with the command python -m pytest -v. The terminal output reported macOS, Python 3.9.21, pytest 8.4.2, Hypothesis 6.141.1, and 24 collected tests. All 24 tests passed, with no failures, in 0.76 seconds.

Metric	Observed result
Tests collected	24
Passed	24
Failed	0
Execution time	0.76 seconds
Operating system	macOS
Python	3.9.21
pytest	8.4.2
Hypothesis plugin	6.141.1 installed

11.2 Test Coverage by Feature

Feature area	Representative verified scenarios
Dashboard and resources	Dashboard loads; resource summaries and alerts render.
Validation	Invalid age/severity and phone values are rejected; duplicate active patient is rejected.
Priority queue	Priority and specialty assignment, search/filtering, editing, Merge Sort order and stable ties.
Treatment lifecycle	Treatment start, start timestamp, completion, outcome/notes persistence and resource release.
Resource management	Doctor/bed status updates, adding resources, duplicate/busy deletion protection, five equipment categories, inventory update.
Navigation and live display	Separate live page, sidebar navigation, allocation preview updates and start-treatment action.
Export	CSV endpoint and generator-based row processing preserve expected header and records.

11.3 Representative Test Case Results

ID	Scenario	Expected behavior	Observed result
TC-01	Dashboard and resources	Dashboard returns successfully and displays key insights.	Passed
TC-02	Critical alert	Critical-category registration triggers the configured alert.	Passed
TC-03	Priority and specialty assignment	Highest-priority waiting patient is selected and a suitable available doctor is assigned.	Passed
TC-04	Search and severity filter	Matching patient appears; non-matching severity is excluded.	Passed
TC-05	Edit waiting patient	Updated waiting-patient details are displayed.	Passed
TC-06	Invalid age/severity	Invalid values are rejected with validation feedback.	Passed
TC-07	Duplicate active patient	Duplicate active record is rejected.	Passed
TC-08	Complete treatment	Outcome/notes are stored and assigned resources are released.	Passed
TC-09	Resource status updates	Doctor and bed status changes persist and are reflected in responses.	Passed
TC-10	CSV export	CSV response contains the expected patient record.	Passed
TC-11	Treatment history timestamp	History displays the treatment start time.	Passed
TC-12	Phone validation	Invalid phone input is rejected.	Passed
TC-13	Separate live display	Live display route is separate and renders queue information.	Passed
TC-14	Add doctor and bed	Resource creation works through the resources interface.	Passed
TC-15	Duplicate/busy deletion protection	Duplicate resource and unsafe deletion conditions are handled.	Passed
TC-16	Five equipment categories	The inventory categories are available in the resources page.	Passed
TC-17	General inventory update	Equipment quantity/status can be updated.	Passed
TC-18	Live display treatment action	The live display action starts the next treatment workflow.	Passed
TC-19	Equipment assignment/release	Equipment assignment is recorded and released on completion.	Passed
TC-20	Merge Sort	Priority order and stable ties are maintained.	Passed
TC-21	Generator CSV export	Generator-based export preserves the header.	Passed

The table above summarizes representative scenarios from the suite. The full test source contains 24 tests; the displayed summary groups related checks rather than listing every test function individually.

11.4 Interpretation and Limits of the Test Results

The result establishes that the 24 automated test cases passed in the reported local environment. It does not demonstrate complete code coverage, property-based test execution, production security, clinical validity, reliability under load, or correctness for every possible input. No coverage percentage or performance benchmark is claimed because such measurements were not included in the supplied terminal output.

11.5 Security and Reliability Considerations

- Use a strong secret key from environment configuration outside local development.
- Keep server-side validation and parameterized SQL for user-supplied values.
- Use synthetic or authorized data for demonstrations.
- Add authentication, authorization, privacy safeguards and audit controls before any real deployment.
- Test race conditions and database locking if concurrent workers are introduced.

12. RESULTS AND DISCUSSION

12.1 Implemented Capabilities

The supplied implementation and screenshots show a multi-page local application covering patient registration, a priority waiting queue, separate live display, doctor management, bed and equipment inventory, treatment start/completion, history, dashboard analytics and CSV export. The test suite verifies representative workflow behavior and resource release.

12.2 Demonstration State in the Screenshots

The dashboard screenshot shows one waiting patient, zero critical cases, two available doctors, three available beds, one allocated bed, an average wait of approximately 30.7 minutes, and zero treated-today cases at the moment captured. The resource inventory view shows four beds in total with one allocated, six wheelchairs with one allocated, eight oxygen cylinders, five patient monitors, two ventilators and six IV infusion pumps. These are snapshot values from the demonstration database, not general performance results.

12.3 Automated Quality Result

The reported test suite passed 24 out of 24 tests in 0.76 seconds. This provides a repeatable baseline for the tested routes and helpers and supports the report's claims about priority ordering, resource lifecycle handling, validation, navigation and CSV generation.

12.4 Performance Evaluation

The algorithmic analysis gives Merge Sort a typical O(n log n) time cost and O(n) auxiliary storage. No separate synthetic-data benchmark was supplied, so the report does not claim measured queue latency, throughput, or scaling performance. A future benchmark should separate sorting time from database retrieval, template rendering and response generation, and report the dataset size, environment, repeated runs and median.

12.5 Discussion

CareQueue demonstrates how computational thinking can be applied to a workflow problem. Decomposition makes the workflow easier to understand, abstraction keeps the data model manageable, Merge Sort provides a custom divide-and-conquer algorithm, and deterministic ordering makes the queue behavior reproducible. Transaction-aware updates help keep patient and resource status consistent during treatment start and completion. Automated tests provide evidence for selected behaviors, while the screenshots document the visible interface.

13. LIMITATIONS AND FUTURE SCOPE

13.1 Limitations

- Severity categories are operator-entered demonstration labels and are not validated clinical triage rules.
- The application is a local educational prototype, not a validated hospital information system.
- The displayed dashboard metrics depend on the local demonstration database and the moment of capture.
- The supplied test result covers 24 test cases but does not establish full coverage or all possible edge-case correctness.
- No measured load/performance benchmark is included.
- Production authentication, role-based permissions, multi-user deployment, hospital integration and formal audit logging are not established.
- Equipment suggestions are keyword-based demonstration rules and require human confirmation; they are not medical recommendations.
- The CSV response is buffered, even though formatted rows are yielded by a generator.

13.2 Future Enhancements

- Add secure authentication, role-based permissions and audit logs.
- Improve validation, privacy controls, secret management and error reporting.
- Add explicit property-based tests if desired and measure test coverage with pytest-cov.
- Configure static analysis and linting tools and document their actual results.
- Add Docker-based reproducible deployment and CI workflows if required by the course.
- Benchmark queue ordering and page response under controlled synthetic dataset sizes.
- Add concurrency tests and strengthen allocation safeguards before considering multiple workers.
- Integrate with authorized clinical systems only under appropriate security, privacy and clinical governance.

14. CONCLUSION

CareQueue demonstrates a simplified emergency-department workflow through patient registration, deterministic priority ordering, doctor and bed allocation, equipment inventory management, treatment completion and history reporting.

The application uses Python, Flask and SQLite, with HTML, CSS and JavaScript templates for the user interface. Its implementation connects computational thinking with practical software development by using decomposition, abstraction, a custom Merge Sort routine, greedy-style resource selection, rule-based suggestions, relational persistence, generators and automated testing.

The supplied local test run collected 24 tests and passed all 24 in 0.76 seconds. The screenshots document the dashboard analytics, queue management, patient registry, doctor directory, bed and equipment inventory, live display and treatment history. These results support the conclusion that the tested prototype functions as an academic demonstration of the specified workflow. They do not establish clinical validity or production readiness. Further security, privacy, performance and concurrency work would be necessary before any real-world use.

REFERENCES

1. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
2. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
3. Python Software Foundation. sqlite3 — DB-API 2.0 interface for SQLite. https://docs.python.org/3/library/sqlite3.html
4. pytest Development Team. pytest Documentation. https://docs.pytest.org/
5. Hypothesis Development Team. Hypothesis Documentation. https://hypothesis.readthedocs.io/
6. CareQueue project source code and tests supplied for this report: app.py, templates/, and tests/test_app.py.